

Mise en place d'une architecture distribuée pour l'analyse de logs web

HDFS, Batch, Streaming et NoSQL avec Docker

Matthieu PELINGRE

IDMC / M2SDD

7 février 2026

Plan de la présentation

- 1 Contexte et objectifs
- 2 Architecture technique
- 3 Traitements Batch (le passé)
- 4 Traitements Streaming (le présent)
- 5 Bilan

Scénario :

- Site e-commerce (Cosmétiques).
- Fort volume de trafic générant des logs serveurs (`web_server.log`).

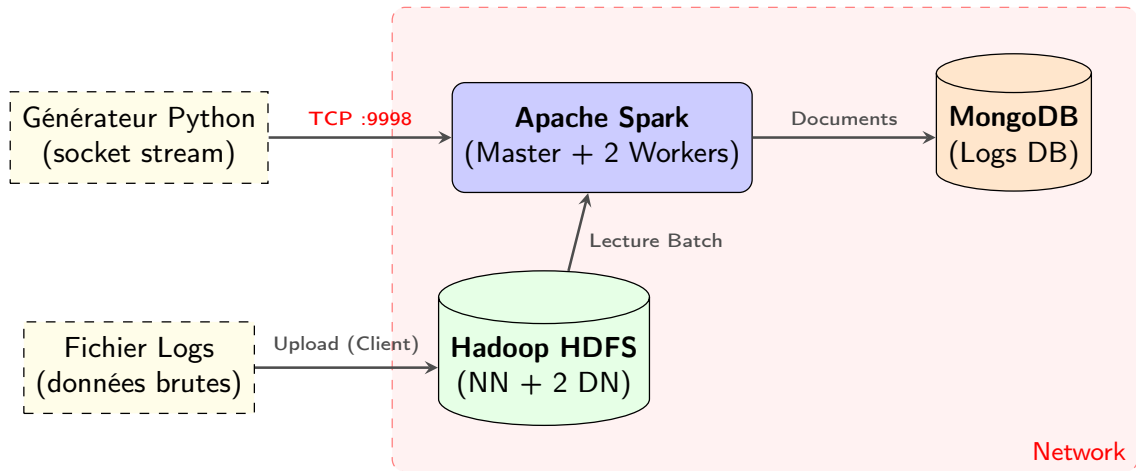
Besoin :

- Transformer des données brutes en informations décisionnelles.
- Besoin de scalabilité (Big Data) et de réactivité (Temps réel).

Objectifs Techniques

- 1 **Stocker** (HDFS).
- 2 **Traiter** (Spark Batch).
- 3 **Surveiller** (Spark Streaming).
- 4 **Persister** (MongoDB).

Architecture déployée (Docker)



Automatisation

- **Batch** : conteneur `hdfs-client` éphémère. Attend la fin du démarrage du NameNode, puis téléverse les données.
- **Stream** : service data-generator résilient (boucle infinie + reconnexion automatique).

Qualité de code

- Création d'un module `utils.py` partagé.
- Centralise la logique de parsing : Regex pour CLF (*Common Log Format*).
- Distribué aux workers via l'option `-py-files`.

Approche MapReduce avec PySpark

- Lecture directe depuis HDFS : `hdfs://namenode:9000/...`
- Parsing des lignes via `utils.parse_log_line()` → RDD

A. Répartition codes HTTP

- RDD en cache pour :
 - Comptage total.
 - Comptage par code HTTP.
- Conversion du RDD en DataFrame.
- **Enrichissement** : Jointure avec descriptions.
- Calcul de fréquence.
- Sauvegarde MongoDB (overwrite).

B. Top 10 IPs

- Comptage par adresse IP.
- Utilisation de `takeOrdered(10)`.
- Conversion du RDD en DataFrame.
- Sauvegarde MongoDB (overwrite).

Résultats Batch (codes HTTP)

Affichage console (Master Spark)

```
1 Demarrage du traitement Batch : AnalyzeLogsCodesHTTP
2 Total des requetes analysees : 4369067
3 Resultats : Frequence des codes HTTP
4 +----+-----+-----+-----+-----+
5 |code|count |description          |frequency|
6 +----+-----+-----+-----+-----+
7 |200 |874321|Succes de la requete  |0.2001   |
8 |403 |874062|Acces refuse          |0.2001   |
9 |404 |873765|Ressource non trouvee |0.2       |
10 |301 |873596|Redirection           |0.2       |
11 |500 |873323|Erreur interne du serveur|0.1999   |
12 +----+-----+-----+-----+-----+
13
14 Sauvegarde dans la collection 'batch_http_codes'
15 Traitement termine avec succes.
```

Résultats Batch (codes HTTP)

Document stocké (batch_http_codes)

```
1 [
2   {
3     _id: ObjectId('697b70bcf6cb610c2cded206'),
4     code: 404,
5     count: 873765,
6     description: 'Ressource non trouvée',
7     frequency: 0.2
8   },
9   {
10    _id: ObjectId('697b70bcf6cb610c2cded207'),
11    code: 500,
12    count: 873323,
13    description: 'Erreur interne du serveur',
14    frequency: 0.1999
15  },
16  ...
```

Résultats Batch (top 10 IPs)

Affichage console (Master Spark)

```
1 Resultats : Top 10 des adresses IP les plus actives
2 +-----+-----+
3 |ip           |count|
4 +-----+-----+
5 |192.168.0.250|4620 |
6 |192.168.1.52 |4571 |
7 |192.168.1.243|4558 |
8 |192.168.0.32 |4553 |
9 |192.168.2.52 |4534 |
10|192.168.0.132|4531 |
11|192.168.3.82 |4529 |
12|192.168.3.154|4524 |
13|192.168.0.101|4522 |
14|192.168.1.49 |4521 |
15+-----+-----+
16
17 Sauvegarde dans la collection 'batch_top_ips'
```

Résultats Batch (top 10 IPs)

Document stocké (batch_top_ips)

```
1 [
2   {
3     _id: ObjectId('697b7107bdbce136d4f6d235'),
4     ip: '192.168.0.132',
5     count: 4531
6   },
7   {
8     _id: ObjectId('697b7107bdbce136d4f6d237'),
9     ip: '192.168.3.82',
10    count: 4529
11  },
12  {
13    _id: ObjectId('697b7107bdbce136d4f6d239'),
14    ip: '192.168.3.154',
15    count: 4524
16  },
17  ...
```

Génération : Si un client se connecte au socket TCP, le data-generator lit en boucle le fichier de logs et envoie un à un les logs ayant le même timestamp. Il attend jusqu'au prochain timestamp et recommence jusqu'à déconnexion du client.

Objectif : Détecter des pics d'erreurs (404/500) *pendant* qu'ils se produisent.

Problématiques Big Data

- ❶ **Event Time** : Utiliser l'heure du log, pas l'heure de la machine.
- ❷ **Données en retard** : Gérer les latences réseau.

Solution : Utilisation du **Watermarking** sur les timestamps des logs (tolérance de 1 min).

Utilisation de **Tumbling Windows** (Fenêtres glissantes) de 10s.

```
1 windowed_counts = errors_df \  
2     .withWatermark("timestamp", "1 minute") \  
3     .groupBy(  
4         window(col("timestamp"), "10 seconds"),  
5         col("code")  
6     ) \  
7     .count()
```

Exemple : "Entre 15 : 19 : 50 et 15 : 20 : 00 → 1372 erreurs 500".

- ➊ **Connexion** au socket TCP et récupération des données dans un **DataFrame**.
- ➋ **Expression régulière** (Spark SQL) pour extraire le timestamp et le code HTTP.
- ➌ **Filtre par code HTTP** ("404", "500").
- ➍ **Watermarking** d'une minute pour les données en retard.
- ➎ **Comptage** en regroupant les timestamps par tranche de 10s (pour tester) et par code.
- ➏ Output mode : **UPDATE**.
- ➐ Sauvegarde dans MongoDB (append).

Problème : Éviter les doublons lors des mises à jour de fenêtres.

Solution : **UPSERT** MongoDB avec clé composite.

NB. On reste en UPDATE sur l'output Spark.

```
1 .mode("append") \  
2 .option("operationType", "replace") \  
3 .option("idFieldList", "start_time,end_time,code")
```

- **Clé unique** : début + fin + code HTTP.
- Garantit la consistance des données.

Bilan Technique :

- ✓ Architecture 100% conteneurisée.
- ✓ Pipeline complet : ingestion → traitement → persistance.
- ✓ Problèmes de synchronisation et de doublons résolus.

Perspectives :

- Tester sur des **données et flux réels**.
- Remplacer le socket TCP par **Kafka**.
- Connecter **Grafana** ou **Metabase** pour faire de la BI.

Merci pour votre attention.